Try simplifying

Across your system prompt, skills, and CLAUDE.md files, you may need to simplify just like we did. We rolled out a new command called `claude doctor,` which will help you do this automatically as well. For more details on prompting more advanced models specifically, check out our Fable field guide.

This article was written by Thariq Shihipar, member of technical staff, Anthropic.

FRED TALKS

5th September 2026

CONTENTS

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · SOURCE

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. Its true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures is a very different thing from traditional infrastructure. All of these properties are critical to large web companies, and are what drove the adoption of horizontally scalable systems like Hadoop, Cassandra, Voldemort, etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a "big data" debate, I am firmly in the "pro-" camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn't true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive N-1 failures, and if the machines fail *independently*, then the probability of losing a particular piece of data must be P^N. So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Applying this to your context

Pulling this all together, what does this look like when you assemble your context?

System Prompt

A system prompt is heavily tied to the product context. It tells Claude what product it's operating in and what it's doing. For Claude Code, you will likely never modify this, but if you are building your own agent harness, this is where you should spend a lot of time.

CLAUDE.md

Keep your CLAUDE.md lightweight and briefly describe what your repo is for, but spend most of the tokens on gotchas inside of the codebase. For example, you may organize your code to keep types in one monolithic file and nowhere else. Avoid stating 'the obvious' things Claude should know by looking at your file system or your repo.

Use progressive disclosure heavily, for example if you have several unique instructions on how to verify your work, create a verification skill and reference it from your CLAUDE.md.

Skills

Think of skills as lightweight guides to let Claude find information when needed. Avoid making them overconstrained, except in highly important areas.

For long skills, try and use progressive disclosure as much as possible- divide it into many files and split them out.

It's best when skills encode particular opinions, knowledge, or best practices that are particular to you, your team, or product.

References

You can @ mention files to include them as references. References allow Claude to refer to in-depth information about the current plan.

This might be in specs files, mockups, or even entire codebases. Generally you should prefer files that are in code as it provides clear, high-fidelity instructions to Claude in a language it knows very well. For example, a HTML mockup of a design will generally produce better results than a description of the design or a screenshot.

Then: Repeat yourself

Now: Simple tool descriptions

Earlier Claude models could sometimes need repeated instructions or be more likely to listen to instructions at the end of their context window than at the start. This meant our system prompt would sometimes have references to tools in the main system prompt as well as instructions in the tool description.

We found we could delete these repeat examples and put instructions on how to use tools in the tool descriptions rather than the system prompt.

Then: Memory in CLAUDE.md files

Now: Auto-memory

We used to encourage users to save things to Claude's memory, by using the # hotkey to write to their CLAUDE.md automatically. Instead, Claude now automatically saves memories that are relevant to the work and to you.

Then: Simple specs

Now: Rich references

In plan mode, Claude Code has heavily relied on markdown files with plans. Storing these files as plans helped Claude refer to them when needed. Another similar best practice was to store specs in the codebase for Claude to refer to while working across longer projects.

But we've found that Claude can handle increasingly more complicated references. Instead of simple markdown files, Claude can reference HTML artifacts created by our new artifacts feature.

You may also give Claude references in the form of code. A spec may also be a detailed test suite, or a function in a different codebase that Claude might port.

Rubrics are another form of references. Rubrics allow Claude to try and verify your taste in a particular field (e.g. what does a good API design look like) by using dynamic workflows and spinning up verifier agents with those rubrics.

Where is the flaw in the reasoning? Is it the dreaded Hadoop single points of failure? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a fantastic paper that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is "cluster aware" is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn't even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind

of “semi-failures” are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn’t have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let’s discuss some real issues. We had a bug in Kafka recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk error, and bringing down the corrupt node is the right behavior—it shouldn’t happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is this Linux bug which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortums for major failures and these are a real wealth of failures in systems that aren’t supposed to fail. This paper has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can’t easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

Then: Give Claude examples

Now: Design interfaces

The number one rule for tool usage was to give Claude examples on how to use them. With our newest models, we’ve found that giving examples actually constrains them to a certain exploration space.

Instead of using examples, think more about the design of your tools, scripts and files- what parameters does Claude have and how can they be more expressive?

For example, in the Todo tool example, just listing status as an enumeration between pending, in_progress, and completed, hints to Claude about how to use it. The instruction on keeping one item in_progress helps define our requested behavior.

Then: Put it all upfront

Now: Use progressive disclosure

Because Claude Code was focused on coding, our system prompt included detailed information on how to do code review and verification. These were not always needed, but when they were, it was crucial information.

Since then, Claude Code has gotten very competent at using progressive disclosure- loading the right context at the right times. For example, we moved verification and code review into their own skills that Claude Code could selectively call.

But progressive disclosure is not just for skills, we also use it for tools. Some of our tools are ‘deferred loading,’ which means the agent must search for their full definitions using ToolSearch before using them. This allows us to have more tools (such as our Task tools) that don’t take up context until they’re needed.

The same can be applied to your own CLAUDE.md and Skill.md files. A common myth is that you want to make these a central repository for every known practice that you *might* run into, because Claude would not find it otherwise. Instead, consider having a tree of files that can be loaded at the right time.

The new rules of context engineering for Claude 5 generation models

CLAUDE.COM · 09 AUG 2026 · SOURCE

Then and now

There were a number of previous context engineering best practices that had become myths. Including:.

Then: Give Claude rules

Now: Let Claude use judgement

When we first rolled out Claude Code, we needed to be sure that Claude avoided worst case scenarios, such as deleting files. This meant we would give particularly strong guidance that might not always be true, For example, in the system prompt we used to say:

In code: default to writing no comments. Never write multi-paragraph docstrings or multi-line comment blocks — one short line max. Don't create planning, decision, or analysis documents unless the user asks for them — work from conversation context, not intermediate files.

But for a certain subset of prompts, this guidance would be wrong. In the case of documentation, the user may have their own preferences, or specific parts of very complex code might need multi-line comment blocks.

Still, without these guardrails for older models, the comments Claude wrote would be incorrect in many cases and we had to accept this tradeoff. But newer models have better judgement and can handle these decisions well without explicit rules.

In the new system prompt we say: *Write code that reads like the surrounding code: match its comment density, naming, and idiom.*

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of from-scratch development in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's DynamoDB. When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released a feature comparison checklist. The checklist was unfair in many ways (as these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when i see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break each of these guarantees. I really like this paper that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see this great blog post on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims–by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a "NoSQL vendor" I think it is reasonable to ask them to provide hard information on customer outages. They don't need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to

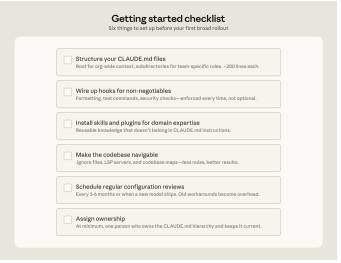
Bottoms-up adoption generates enthusiasm but can fragment without someone to centralize what works. You need to have an individual or a team assemble and evangelize the right Claude Code conventions (such as a standardized CLAUDE.md hierarchy or a curated set of skills and plugins). Without that work, knowledge will stay tribal and adoption will plateau.

In large organizations, especially those in regulated industries, governance questions come up early, such as: who controls which skills and plugins are available, how do you prevent thousands of engineers from independently rebuilding the same thing, how do you make sure AI-generated code goes through the same review process as human-generated code? To address these early on, we suggest starting with a defined set of approved skills, required code review processes, and limited initial access, and expand as confidence builds.

We've observed the smoothest deployments at organizations that establish cross-functional working groups early by bringing together engineering, information security, and governance representatives to define requirements together and build a rollout roadmap.

Applying these patterns to your organization

Claude Code is designed around conventional software engineering environments where engineers are the primary codebase contributors, the repo uses Git, and code follows standard directory structures. Most large codebases fit this mold, but non-traditional setups such as game engines with large binary assets, environments with unconventional version control, or non-engineers contributing to the codebase require additional configuration work. Our guidance assumes a conventional setup and the patterns we've described have worked across many of our customers. Any remaining complexity requires judgment specific to your codebase, tooling, and organization. That's where Anthropic's Applied AI team works directly with engineering teams to translate these patterns into your organization's specific requirements.



Acknowledgements: *Special thanks to Alon Krifcher, Charmaine Lee, Chris Concannon, Harsh Patel, Henrique Savelli, Jason Schwartz, Jonah Dueck and Kirby Kohlmorgen from Anthropic's Applied AI team for sharing their experience deploying Claude Code at scale, and to Amit Navindgi at Zoox for providing feedback on this article.*

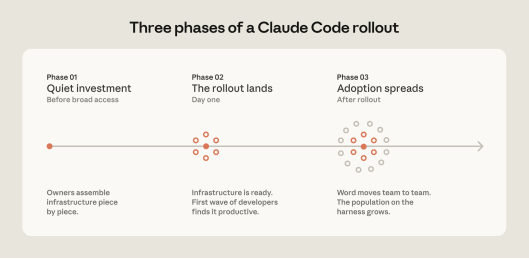
Skills and hooks built to compensate for specific model limitations, whether in the model's reasoning or in Claude Code's own tooling, become overhead once those limitations no longer exist. A hook that intercepted file writes to enforce p4 edit in a Perforce codebase, for example, became redundant once Claude Code added native Perforce mode.

Teams should expect to do a meaningful configuration review every three to six months, but it's also worth doing one whenever performance feels like it's plateaued after major model releases.

Assigning ownership for Claude Code management and adoption

Technical configuration alone doesn't drive adoption. The organizations that got it right invested in the organizational layer, too.

The rollouts that spread fastest had a dedicated infrastructure investment before broad access. A small team, sometimes even just one person, wired up the tooling so Claude already fit developer workflows when they first touched it. At one company, a couple of engineers built a suite of plugins and MCPs that were available on day one. At another, an entire team focused on managing AI coding tools had the infrastructure in place before the rollout began. In both cases, developers' first experience was productive rather than frustrating, and adoption spread from there.



The teams doing this work today tend to sit under developer experience or developer productivity, which is typically the function responsible for onboarding new engineers and building developer tooling. An emerging role in several organizations is an agent manager: a hybrid PM/engineer function dedicated to managing the Claude Code ecosystem. For organizations without a dedicated team, the minimum viable version is a DRI: one person with ownership over the Claude Code configuration, the authority to make calls on settings, permissions policy, the plugin marketplace, and CLAUDE.md conventions, and the responsibility to keep them current.

achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn't clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the "big data" or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, Confluent offers Kafka as a service with a contractural uptime SLA. Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

How Claude Code navigates large codebases

CLAUDE · 18 MAY 2026 · SOURCE

Claude Code is running in production across multi-million-line monorepos, decades-old legacy systems, distributed architectures spanning dozens of repositories, and at organizations with thousands of developers. These environments present challenges that smaller, simpler codebases don't, whether that's build commands that differ across every subdirectory or legacy code spread across folders with no shared root.

This article covers the patterns we've observed that have led to successful adoption of Claude Code at scale. We use "large codebase" to refer to a wide range of deployments: monorepos with millions of lines, legacy systems built over decades, dozens of microservices across separate repositories, or any combination of the above. That also includes codebases running on languages that teams don't always associate with AI coding tools, such as C, C++, C#, Java, PHP. (Claude Code performs better than most teams expect it to in those cases, particularly as of

recent model releases.) While every large codebase deployment is shaped by its specific version control, team structure, and accumulated conventions, the patterns here generalize across them and are a good starting point for teams considering adopting Claude Code.

Claude Code navigates a codebase the way a software engineer would: it traverses the file system, reads files, uses grep to find exactly what it needs, and follows references across the codebase. It operates locally on the developer's machine and doesn't require a codebase index to be built, maintained, or uploaded to a server.

RAG-powered AI coding tools work by embedding the entire codebase and retrieving relevant chunks at query time. At large scale, those systems can fail because embedding pipelines can't keep up with active engineering teams. By the time a developer queries the index, it reflects the codebase as it previously existed weeks, days, or even hours before. Retrieval then returns a function the team renamed two weeks ago, or references a module that was deleted in the last sprint, with no indication that either is out of date.

Agentic search avoids those failure modes. There's no embedding pipeline or centralized index to maintain as thousands of engineers commit new code. Each developer's instance works from the live codebase.

But the approach has a tradeoff: it works best when Claude has enough starting context to know where to look. This means the quality of Claude's navigation is shaped by how well the codebase is set up, layering context with CLAUDE.md files and skills. If you ask it to find all instances of a vague pattern across a billion-line codebase, you'll hit context-window limits before the work begins. Teams that invest in codebase setup see better results.

The harness matters as much as the model

One of the most common misconceptions about Claude Code is that its capabilities are solely defined by the model used. Teams focus on a model's benchmarks and how it performs on test tasks. In practice, the ecosystem built around the model—the harness—determines how Claude Code performs more than the model alone.

The harness is built from five extension points—CLAUDE.md files, hooks, skills, plugins, and MCP servers—each serving a different function. The order in which teams build them matters, as each layer builds on what came before. Two additional capabilities, LSP integrations and subagents, round out the setup. Below, we explain what each of these components and capabilities do:

- **Using `.ignore` files to exclude generated files, build artifacts, and third-party code.** Committing `permissions.deny` rules in `.claude/settings.json` means the exclusions are version-controlled, so every developer on the team gets the same noise reduction without configuring it themselves. In some codebases, generated files are themselves the subject of development work. Developers who work on code generators can override project-level exclusions in their local settings without affecting the rest of the team.

- **Building codebase maps when the directory structure doesn't do the work.** For organizations where code isn't consolidated in a conventional directory structure, a lightweight markdown file at the repo root listing each top-level folder with a one-line description of what lives there gives Claude a table of contents it can scan before opening files. For codebases with hundreds of top-level folders, this works best as a layered approach: the root file describes only the highest-level structure, and subdirectory CLAUDE.md files provide the next level of detail, loading on demand as Claude moves through the tree. For simpler cases, @-mentioning the specific files or directories Claude should reference can do the same job.

- **Running LSP servers so Claude searches by symbol, not by string.** Grep for a common function name in a large codebase returns thousands of matches and Claude burns context opening files to figure out which matters. LSP returns only the references that point to the same symbol, so the filtering happens before Claude reads anything. Setting this up requires installing a code intelligence plugin for your language and the corresponding language server binary; the Claude Code documentation covers the available plugins and troubleshooting.

One caveat: there are edge cases where even the hierarchical CLAUDE.md approach breaks down, for example codebases with hundreds of thousands of folders and millions of files, or legacy systems on non-git version control. We will address their challenges in future installments of this series.

Actively maintaining CLAUDE.md files as model intelligence evolves

As models evolve, instructions written for your current model can work against a future one. CLAUDE.md files that guided Claude through patterns it used to struggle with may either become unnecessary or actively constraining when the next model ships. For example, a CLAUDE.md rule that tells Claude to break every refactor into single-file changes may have helped an earlier model stay on track but would prevent a newer one from making coordinated cross-file edits it handles well.

Subagents*	Separate Claude instances for specific tasks	When invoked	Splitting exploration from editing, parallel work	Running exploration and editing in the same session

*LSP is accessed through the plugin layer. Subagents are a delegation capability rather than a configured extension point.

Three configuration patterns from successful deployments

How you configure Claude Code for a large codebase depends heavily on how that codebase is structured. Still, three patterns appeared consistently across the deployments we observed.

Making the codebase navigable at scale

Claude's ability to help in a large codebase is bounded by its ability to find the right context. Too much context loaded into every session degrades performance, while too little context leaves Claude to navigate blind. The most effective deployments invest upfront in making the codebase legible to Claude. A few patterns appear consistently:

- **Keeping CLAUDE.md files lean and layered.** Claude loads them additively as it moves through the codebase: root file for the big picture, subdirectory files for local conventions. The root file should be pointers and critical gotchas only; everything else drifts into noise.

- **Initializing in subdirectories, not at the repo root.** Claude works best when it's scoped to the part of the codebase that's actually relevant to the task. In monorepos, this can feel counterintuitive because tooling often assumes root access, but Claude automatically walks up the directory tree and loads every CLAUDE.md file it finds along the way, so root-level context is never lost.

- **Scoping test and lint commands per subdirectory.** Running the full suite when Claude changed one service causes timeouts and wastes context on irrelevant output. CLAUDE.md files at the subdirectory level should specify the commands that apply to that part of the codebase. This works well for service-oriented codebases where each directory has its own test and build commands. In compiled-language monorepos with deep cross-directory dependencies, per-subdirectory scoping is harder to achieve and may require project-specific build configurations.

CLAUDE.md files come first. These are context files that Claude reads automatically at the start of every session: root file for the big picture, subdirectory files for local conventions. They give Claude the codebase knowledge it needs to do anything well. Because they load in every session regardless of the task, keeping them focused on what applies broadly will prevent them from becoming a drag on performance.

Hooks make the setup self-improving. Most teams think of hooks as scripts that prevent Claude from doing something wrong, but their more valuable use is continuous improvement. A stop hook can reflect on what happened during a session and propose CLAUDE.md updates while the context is fresh. A start hook can load team-specific context dynamically so every developer gets the right setup for their module without manual configuration. For automated checks like linting and formatting, hooks enforce the rules deterministically and produce more consistent results than relying on Claude to remember an instruction.

Skills keep the right expertise available on-demand without bloating every session. In a large codebase with dozens of task types, not all expertise needs to be present in every session. Skills solve this through progressive disclosure, offloading specialized workflows and domain knowledge that would otherwise compete for context space and loading them only when the task calls for it. For example, a security review skill loads when Claude is assessing code for vulnerabilities, while a document processing skill loads when a code change is made and documentation needs to be updated.

Skills can also be scoped to specific paths so they only activate in the relevant part of the codebase. A team that owns a payments service can bind their deployment skill to that directory, so it never auto-loads when someone is working elsewhere in the monorepo.

Plugins distribute what works. One challenge with large codebases is that *good* setups can stay tribal. A plugin bundles skills, hooks, and MCP configurations into a single installable package, so when a new engineer installs that plugin on day one, they will immediately have the same context and capabilities as those who have been using Claude already. Plugin updates can be distributed across the organization through managed marketplaces.

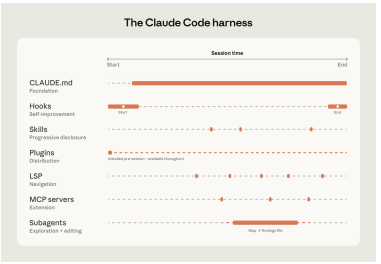
For example, a large retail organization we work with built a skill connecting Claude to their internal analytics platform so that business analysts could pull performance data without leaving their workflow. They distributed it as a plugin before the broad rollout to the business.

Language server protocol (LSP) integrations give Claude the same navigation a developer has in their IDE. Most large-codebase IDEs already have an LSP running, powering "go to definition" and "find all references." Surfacing this to Claude gives it symbol-level precision: it can follow a function call to its definition, trace references across files, and distinguish between

identically named functions in different languages. Without it, Claude pattern-matches on text and can land on the wrong symbol. One enterprise software company we worked with deployed LSP integrations org-wide before their Claude Code rollout, specifically to make C and C++ navigation reliable at scale. For multi-language codebases, this is one of the highest-value investments.

MCP servers extend everything. MCP servers are how Claude connects to internal tools, data sources, and APIs that it can't otherwise reach. The most sophisticated teams built MCP servers exposing structured search as a tool Claude can call directly. Others connect Claude to internal documentation, ticketing systems, or analytics platforms.

Subagents split exploration from editing. A subagent is an isolated Claude instance with its own context window that takes a task, does the work, and returns only the final result to the parent. Once the harness is in place, some teams spin up a read-only subagent to map a subsystem and write findings to a file, then have the main agent edit with the full picture.



Claude Code’s extension layer at a glance.

The table below summarizes what each component does, when it loads, and the most common mistakes we see with each:

Component	What it is	When it loads	Best for	Common confusion
CLAUDE.md	Context file Claude reads automatically	Every session	Project-specific conventions, codebase knowledge	Using it for reusable expertise that belongs in a skill
Hooks	Scripts that run at key moments	Triggered by events	Automating consistent behavior, capturing session learnings	Using prompts for things that should run automatically
Skills	Packaged instructions for specific task types	On demand, when relevant	Reusable expertise across sessions and projects	Loading everything into CLAUDE.md instead
Plugins	Bundled skills, hooks, MCP configs	Always available once configured	Distributing a working setup across the org	Letting good setups stay tribal
Language server protocol (LSP)*	Real-time code intelligence via language specific servers	Always available once configured	Symbol-level navigation and automatic error detection in typed languages	Assuming that it's automatic
MCP servers	Connections to external tools and data	Always available once configured	Giving Claude access to internal tools it can't otherwise reach	Building MCP connections before the basics are working